Q1) a)

Align()

* inputs: match score
mismatch score
gap penalty
slippage gap penalty
sequences S and T of lengths m and n

- Initialize scoring matrix A and pointer matrix P of size (m+1 x n+1)

- Fill in first row of pointer matrix P with "L" (left)
- " " " column " " " " " "T" (top)
- " " " item (row, col = 0) with "X" to signify end

- Fill in first row of scoring matrix A with (gap penalty) x j (j = col index)
- " " " column " " " " " (gap penalty) x i (i = row index)

- For every row i until m: (i and j from 1 to m,n inclusif)
For " column j " n: (not 0)

for ↖
if: S[i-1] = T[j-1]:
diagonal score = A[i-1][j-1] + match score
else:
diagonal score = A[i-1][j-1] + mismatch score

for ↑
if: S[i-1] = S[i-2]
top score = A[i-1][j] + slippage gap penalty
else:
top score = A[i-1][j] + gap penalty

for ←
if: T[j-1] = T[j-2]
left score = A[i][j-1] + slippage gap penalty
else:
left score = A[i][j-1] + gap penalty

A[i][j] = maximum (diagonal score , top score , left score)

if A[i][j] = diagonal score :
    P[i][j].append("D")   D for diagonal

if A[i][j] = left score
    P[i][j].append("L")   L for left score

if A[i][j] = top score
    P[i][j].append("T")   T for top score

Backtrack function call (recursive)

* Backtrack (recursive)
inputs S, T, Pointer matrix P, Salignment, Talignment,
Alignments found list

if i=0 and j=0
Alignments found.add(inverse of Salignment and Talignment)
return (exits the function)

if "D" is in P[i][j]:
add S[i-1] to S alignments
" T[j-1] " T alignments
Backtrack with new S alignment, T alignment, i-1, j-1
remove S[i-1] from S alignment
" T[j-1] " T alignment

if "L" in P[i][j]
add "-" gap to S alignment
" T[j-1] to T alignment
Backtrack with new S alignment T alignment, i, j-1
remove "-" gap from S alignment
" T[j-1] from T alignment

if "T" in P[i][j]
add "-" gap to T alignment
add S[i-1] to S alignment
Backtrack with new S alignment T alignment, i-1, j
remove "-" gap from T alignment
remove S[i-1] from S alignment

b)

I get a score of 119 for the small alignment task and 215 for the full alignmeat task.

I submitted the 2 output alignments with scores on mycourses.

Question 2)

* S = APPLE
T = CAPE
c = -1
match score = 1
mismatch score = -1

		O	A	P	P	L	E	: S
	O	0	-1	-2	-3	-4	-5	
	C	-1	-1	-2	-3	-4	-5	
	A	-2	0	-1	-2	-3	-4	
	P	-3	-1	1	0	-1	-2	
	E	-4	-2	0	0	-1	0	
	: T							

Sol 1:

_ A P P L E : S
C A _ P _ E : T
score: -1 +1 -1 +1 -1 +1 = 0 ✓

same here

Sol 2:

_ A P P L E : S
C A P _ _ E : T
score: -1 +1 +1 -1 -1 +1 = 0 ✓

and here

Question 3)

T A = 5 } 2 sequences
A T = T }

score = −1 − 1 = −2

	0	T	A
0	0	−1	−2
A	−1	−1	0
T	−2	0	−1

_ T A
A T _

or

T A _
_ A T

scores = −1 + 1 − 1 = −1

both optimal for linear gap penalty of c = −1

with affine gap penalty, the 2 solutions here will have a score of

$$-2.5 + 1 - 2.5 = -5 + 1 = -4$$

and T A / A T will have a score of −2 making it the optimal solution with affine penalty.

so $optLin = \left\{ \begin{matrix} _\ T\ A \\ A\ T\ _ \end{matrix} \text{ and } \begin{matrix} T\ A\ _ \\ _\ A\ T \end{matrix} \right\}$

$optAff = \left\{ \begin{matrix} T & A \\ A & T \end{matrix} \right\}$

and $optLin \cap optAff = \emptyset$

Q4) given S, T we want to find X (the longest common subsequence)

We can use SmithWaterman Algorithm but modify it slightly

$$X_{ij} = \max \begin{pmatrix} X_{i-1,j-1} + M(s_i, t_j) \\ X_{i,j-1} + C \\ X_{i-1,j-1} + C \end{pmatrix}$$

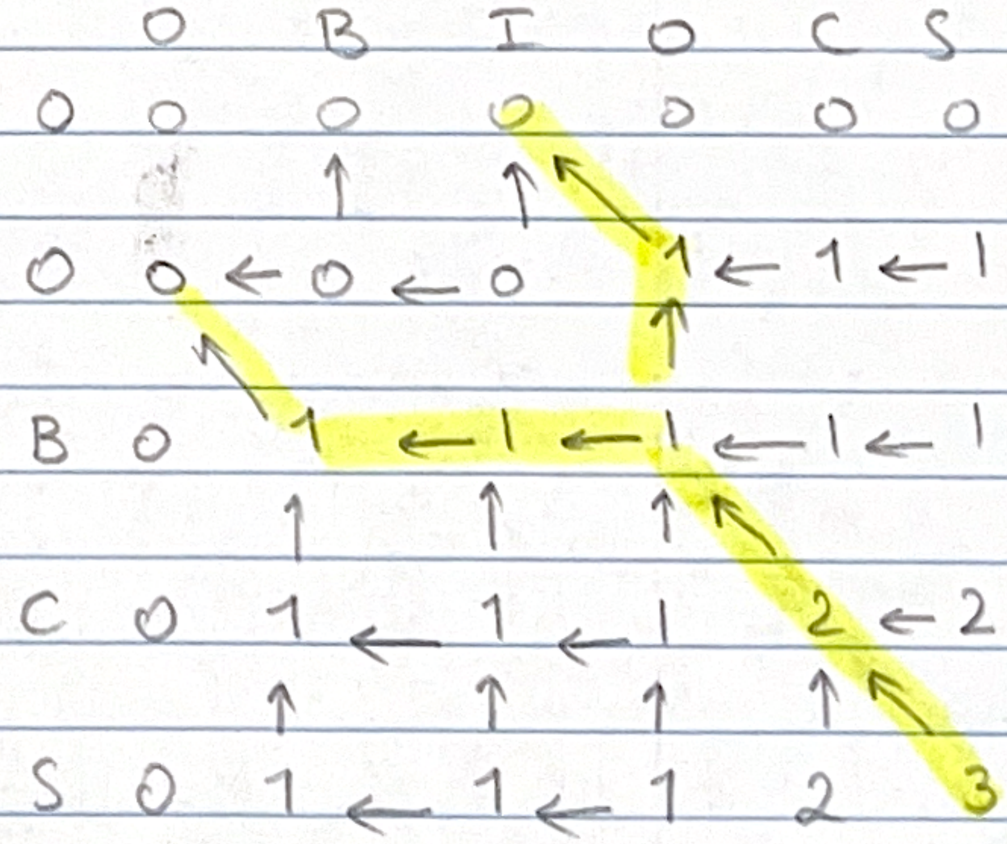
$M(S_i, t_j) = 1$ if $S_i = T_j$

- but we do not want to allow mismatches. so we can set $M(S_i, t_j) = -\infty$ to do that (or just ignore mismatches and never have ↖ diag pointer for mismatches)

- we can either ignore c "gap penalty" or set it to zero to create X

During backtracking, we only consider S[i] & T[j] when there is a diagonal pointer "↖" for matches. ↖ can never correspond to a mismatch since we donot allow mismatches.

Example: S = BIOCS
T = OBCS

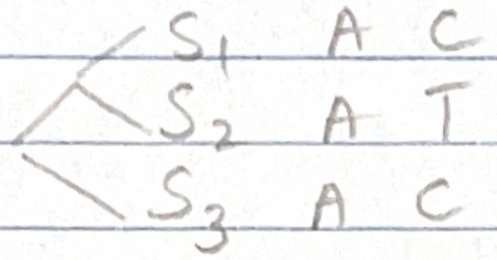


we need to start from argmax of X until we don't have ↖ anymore

X = BCS and OCS ✓

Q5) c = -1
match = 1
mismatch = -1

Consider the sequences S_1-S_3 and the tree provided:



Align S_1 & S_2 which are siblings (NW)

$$Align(S_1, S_2) = \begin{matrix} A \\ A \end{matrix} \quad \begin{matrix} C \\ T \end{matrix}$$

now we have

Ancestor (S_1, S_2) : $\begin{matrix} A \\ A \end{matrix} \quad \begin{matrix} C \\ T \end{matrix}$

S_3 : ACT

$$Align(Ancestor(S_1, S_2), S_3) = \begin{matrix} A & C & - \\ A & T & - \\ A & C & T \end{matrix} \quad \begin{matrix} 0 \\ -1 \end{matrix} \quad 1 = 0$$

(score)

The above alignment is not optimal.
The optimal " is :

$$\begin{matrix} A & C & - \\ A & - & T \\ A & C & T \end{matrix} \quad \begin{matrix} -1 \\ 1 \end{matrix} \quad 1 = 1 \quad \text{and} \quad 1 > 0$$

(score)

The algorithm did not produce the optimal alignment because it doesn't align all sequences at once.
Instead it aligns and fixes sister branches in the phylogenetic tree.

q = AGCAAGTCCGA ω = 4 Δ = 5

*, **, ***(1) and ***(2) are short perfect matches

$$M = \begin{array}{c} \\ A \\ C \\ G \\ T \end{array} \begin{bmatrix} A & C & G & T \\ +1 & -2 & -1 & -2 \\ -2 & +1 & -2 & -1 \\ -1 & -2 & +1 & -2 \\ -2 & -1 & -2 & +1 \end{bmatrix}$$

Q6) a

D = GGGCATGTCCTCATAGCGAGCCCCGTAGGAGTAAGTCAGATTGGGGTCCCCCGT

(i

Further examine *:

D: AGTAAGTCAGA

q: AGCAAGTCCGA

score: 5 4 3 4 5 3 4 5

Further examine **:

D: AGTAAGTCAGA

q: AGCAAGTCCGA

score: 6 5 4 5 4 2 3 4

Further examine ***(1)

D: GGCATGTCCTC

q: AGCAAGTCCGA

score: 4 5 4 3 2 4 2 0

Further examine ***(2):

D: TGGGGTCCCC

q: GCAAGTCCGA

score: -2 0 2 3 4 2 0

Δ reached (4-5 = -1) so stop 0 ≥ 0 (inclusif)

Alignments:

*: D: AGTAAGTCAGA (SPM)
q: AGCAAGTCCGA
score: 1 1 -1 1 1 1 1 -2 1 1 = 6

**: D: AGTAAGTCAGA (SPM)
q: AGCAAGTCCGA
score: 1 1 -1 1 1 1 1 1 -2 1 1 = 6

***(1): D: GGCATGTCCTC (*SPM)
q: AGCAAGTCCGA
score: -1 1 1 1 -2 1 1 1 1 -2 -2 = 0

***(2): D: TGGGGTCCCCC (Δ reached, SPM)
q: GCAAGTCCGA
score: -1 1 1 1 -2 -2 = 0 4

do not report these cuz Δ reached. So only reporting highest score

b) Since $w = 4$, BLAST can miss smaller short perfect matches that lead to good alignments. ingeneral, we increase sensitivity with smaller w at the cost of time and resources.

Here we missed the word GTC that aligns as such:

```
q : A G C A A G T C C G
D : A G C G A G C C C G
    | | | -1 | | -1 | | |
```

score = 6